BLG422E: Computer Networks - Homework

Corresponding TA: T. Tolga Sarı(sarita@itu.edu.tr)

Q1) Stop-and-Wait & Go-Back-N Protocols in OMNET++

Implement two reliable data protocols: Stop-and-Wait and Go-Back-N (Kurose 3.4.3). Use sequence numbers, ACKs, and timers. Stop-and-Wait sends one segment and waits for its ACK (no new data until ACK or timeout). Go-Back-N keeps a window of size N and retransmits from the first unACKed segment on timeout.

You **must** use the provided `Rdt3Network` topology and `RdtPacket` message defined in the appendices (see Appendix A and Appendix B). Do not change gate names or parameter names. Use:

- **Stop-and-Wait**: `windowSize = 1`.
- **Go-Back-N**: `windowSize` > 1.

Log per-flow throughput, average one-way delay, retransmission count, and completion time under several channel loss settings (vary PER of the lossy links). Discuss what happens as you increase PER. Does it always make sense to have highest N for Go-Back-N?

Project Setup & File Placement

Create your OMNET++ project. Under the project root, create `src/rdt/`. Put:

- `Rdt3Network.ned` (Appendix A) into `src/rdt/`,
- `RdtPacket.msg` (Appendix B) into `src/rdt/`.

Add your implementations as:

- `src/rdt/RdtClient.cc`,
- `src/rdt/RdtServer.cc`.

Ensure `src` is both a C++ source folder and a NED source folder (Project Properties/OMNET++/NED Source Folders). Build once so the message compiler generates `RdtPacket_m.h/.cc`. Use the starter `omnetpp.ini` in Appendix C and adapt parameters when experimenting.

Node Behaviour

Use `seqNum`, `ackNum`, and flags consistently on both sides. Moreover, check if a transmitted packet has error using `cPacket::hasBitError()` method.

RdtClient:

- At `requestTime`, send a request packet (`isRequest=true`) to the server.
- Receive data packets, deliver in-order payloads, discard duplicates, and acknowledge every in-order segment using `RdtPacket` (`isAck=true`, with `ackNum` set appropriately).
- Measure and emit the declared signals and statistics defined in the NED file.

RdtServer:

- Upon receiving a request, send the file in segments of size `maxMsgSize` using `RdtPacket` (`isData=true`, `seqNum` field).
- The actual file contents are irrelevant; assume there is a certain amount of data to be transmitted.
- Implement Stop-and-Wait when `windowSize=1` and Go-Back-N when `windowSize=N>1`, using a single timeout timer and proper retransmission logic.
- Update and emit statistics: data packets sent, retransmitted packets, ACKs received, requests received, and corrupted drops.
- Send a FIN packet (`isFin=true`) when the transfer is complete.

Q2) TCP Performance in Dynamic Networks

Replace the app in `tutorials/wireless/Wireless13` with a TCP client/server. Vary the antenna power and collect at least the following metrics:

- **TCP state variables:** `ssthresh` (initial and per-epoch), `cwnd(t)`, retransmission counter, `RTO(t)`, `RTT/RTTVAR(t)`, `MSS`, flight size, duplicate ACK bursts.
- **Performance metrics:** throughput, delivery ratio.
- **Network behavior:** packet loss count/rate and reason, bottleneck queue length(t).
- **PHY/MAC layer:** PHY data rate, effective MAC rate, retransmissions, collisions, average RSSI/SNR.

Export all relevant vectors/scalars and plot the key metrics. **Discuss** how they change with different network parameters. Additional parameter sweeps (e.g., node velocity) are welcome.

Submission & Report

Your archive must include:

- All new and modified files (`.cc`, `.ned`, `omnetpp.ini`) under `q1` and `q2` folders.
- Include the `.vec` and `.sca` files from at least one representative simulation run, so we can verify the collected metrics directly in OMNET++.

Report: concise PDF with **plenty of screenshots and graphs proving you actually ran the simulations**. Include at least:

- OMNET++ run configuration dialog, NED topology, parameter inspectors, and **vector/scalar plot windows**.
- Graphs for all parameter sweeps and measurement metrics.

Submissions missing screenshots, plots, or required files will lose significant points. Incorrect file placement or missing README will also be penalized. **Reproducibility is mandatory.**

Appendix A: (`Rdt3Network.ned`) for Q1

Save as `src/rdt/Rdt3Network.ned`.

```
import ned.DatarateChannel;

simple RdtClient
{
    parameters:
        int maxMsgSize @unit(B) = default(1000B);       // also used as max segment size
        double requestTime @unit(s) = default(0s);

        @signal[dataPkRcvd](type=long);
        @signal[ackPkSent](type=long);
        @signal[corruptedPkDrop](type=long);
        @signal[dataDelay](type=simtime_t);

        @statistic[dataPkRcvd](title="Data packets received"; source=dataPkRcvd; record=count;);
        @statistic[ackPkSent](title="ACK packets sent"; source=ackPkSent; record=count;);
        @statistic[corruptedPkDrop](title="Corrupted packets dropped"; source=corruptedPkDrop; record=count;);
        @statistic[dataDelay](title="One-way data delay"; source=dataDelay; record=mean, histogram;);

    gates:
        input  in;
        output out;
}

simple RdtServer
{
    parameters:
        int maxMsgSize @unit(B) = default(1000B);       // also used as segment size
        int windowSize = default(100);                  // N: max in-flight segments
        int fileSizeBytes @unit(B) = default(100000B);
        double timeout @unit(s) = default(0.2s);

        @signal[dataPkSent](type=long);
        @signal[dataPkRetx](type=long);
        @signal[ackPkRcvd](type=long);
        @signal[reqPkRcvd](type=long);
        @signal[corruptedPkDrop](type=long);

        @statistic[dataPkSent](title="Data packets sent"; source=dataPkSent; record=count;);
        @statistic[dataPkRetx](title="Data packets retransmitted"; source=dataPkRetx; record=count;);
        @statistic[ackPkRcvd](title="ACK packets received"; source=ackPkRcvd; record=count;);
        @statistic[reqPkRcvd](title="Request packets received"; source=reqPkRcvd; record=count;);
        @statistic[corruptedPkDrop](title="Corrupted packets dropped"; source=corruptedPkDrop; record=count;);

    gates:
        input  in;
        output out;
}

channel LossyLink extends ned.DatarateChannel
{
    parameters:
        datarate = default(10Mbps);
        delay = default(10ms);
        per = default(0.1);   // packet error rate 0..1
        ber = default(0);     // bit error rate 0..1
}

network Rdt3Network
{
    submodules:
        client: RdtClient { @display("p=100,200"); }
        server: RdtServer { @display("p=300,200"); }
    connections:
        client.out --> c2s: LossyLink --> server.in;
        server.out --> s2c: LossyLink --> client.in;
}
```

Listing 1: Rdt3Network.ned

Appendix B: (`RdtPacket.msg`) for Q1

Save as `src/rdt/RdtPacket.msg`. Include the generated header `#include "RdtPacket_m.h"` in your C++.

```
packet RdtPacket
{
    int seqNum = -1;      // data segment index (0..), or unused for control
    int ackNum = -1;      // cumulative ACK (largest in-order seq received)

    bool isData = false;    // true => data segment
    bool isAck = false;     // true => ACK
    bool isFin = false;     // true => FIN (file transfer complete)
    bool isRequest = false; // true => file request (from client to server)

    int payloadBytes = 0;   // size of application data carried in this packet
}
```

Listing 2: RdtPacket.msg

Appendix C: Suggested `omnetpp.ini` for Q1

Place this in your `omnetpp.ini` and adjust parameters when experimenting. Record statistics must stay on.

```
[General]
network = Rdt3Network

# Channel parameters (vary these in experiments)
**.s2c.per = ${channelPer = 0.0, 0.1, 0.2}
**.c2s.per = ${channelPer}

**.s2c.datarate = 10Mbps
**.c2s.datarate = 10Mbps

**.s2c.delay = 10ms
**.c2s.delay = 10ms

# Client/server parameters
**.client.maxMsgSize     = 1000B
**.client.requestTime    = 0s
**.server.maxMsgSize     = 1000B
**.server.windowSize     = ${windowSize = 1, 10, 100}
**.server.fileSizeBytes  = 200000B
**.server.timeout        = 0.2s # Experiment different timeout values
```

Listing 3: omnetpp.ini